

Ingeniería del Software II

Práctica #3 – Search-based Testing

Ejercicio 1

Sea el programa `test_me` y el siguiente test suite:

```
def test_me(n: int) -> int:
1:  r:int=0
2:  if n>=0:
3:      i:int=0
4:      while i < n:
5:          if i % 2 ==1:
6:              r=r+i
7:              i=i+1
8:  return r

class TestSuite(unittest.TestCase):
    def test_1(self):
        self.assertEqual(0, test_me(-1000))

    def test_2(self):
        self.assertEqual(0, test_me(0))

    def test_3(self):
        self.assertEqual(0, test_me(1))
```

- Asumiendo un $K=1$ ¿Cuál es el valor de la distancia de branch no normalizada para cada decisión si ejecutamos el test suite?
- ¿Cuál es el cubrimiento de líneas?
- ¿Cuál es el cubrimiento de branches?

Ejercicio 2

Sea el programa `test_me` y el siguiente test suite:

```
def test_me(x: int, y: int) -> bool:
1:  result: bool = False
2:  z: int = 2 * y
3:  if z == x: # c1
4:      if x > y + 10: # c2
5:          result = True
6:  return result

class TestSuite(unittest.TestCase):
    def test_1(self):
        self.assertEqual(0, test_me(0,0))

    def test_2(self):
        self.assertEqual(0, test_me(1,1))

    def test_3(self):
        self.assertEqual(0, test_me(2,2))
```

- a. Completar la tabla con los valores de la distancia de branch no normalizada para cada decisión (tanto true como false) luego de ejecutar **todo** el test suite. El valor de K para la distancia de branch es 0,5.

	c1	c2
distance_true		
distance_false		

- b. ¿Cuál es el cubrimiento de líneas?
- c. ¿Cuál es el cubrimiento de branches?

Ejercicio 3

Sea el siguiente programa:

```

1 def test_me(k: int, j: int) -> int:
2     r: int = 1
3     if k != 0 and k > 0: #C1
4         c: int = k % 3 # k modulo 3
5         i: int = 0
6         while i < c: #C2
7             r = r * j
8             i = i + 1
9     return r

```

y el siguiente test suite:

```

class TestSuite(unittest.TestCase):
    def test_0(self):
        self.assertEqual(1, test_me(-1, 0))
    def test_1(self):
        self.assertEqual(1, test_me(3, 2))

```

- a. ¿Cuál es el valor de la distancia de branch no normalizada para cada decisión si ejecutamos el test suite? Suponer que $K = 5$.

branch	distancia true	distancia false
3: if k != 0 and k > 0:		
6: while i < c:		

- b. ¿Cuál es el cubrimiento de branches que logra el test suite?

Ejercicio 4

Sea el programa `test_me`

```

1 def test_me(k: int, j: int) -> int:
2     r: int = 1
3     if k > 0 and k < 3: #C1
4         i: int = 0
5         while i < k: #C2
6             r = r * j
7             i = i + 1
8     return r

```

y el siguiente test suite:

```
class TestSuite(unittest.TestCase):
    def test_0(self):
        self.assertEqual(1, test_me(-1,0))
    def test_1(self):
        self.assertEqual(1, test_me(3,2))
```

- a. ¿Cuál es el valor de la distancia de branch no normalizada para cada decisión si ejecutamos el test suite? Suponer que $K = 3$.

branch	distancia true	distancia false
3: if k>0 and k<3:		
5: while i<k:		

- b. ¿Cuál es el cubrimiento de branches que logra el test suite?

Ejercicio 5

Sea el siguiente programa:

```
def func(x: int) -> int:
1:   a:int = x + 1
2:   b:int = a * 2
3:   if b > 10:
4:       c:int = b - x
5:   else:
6:       c:int = b + x
7:   d:int = c ** 2
8:   return d
```

- a) Escribir el control flow graph
b) Escribir los dominadores y postdominadores de cada nodo del control flow graph

Ejercicio 6

Sea el siguiente programa:

```
def foo(x: int) -> int:
1:   y: int = 0
2:   while x > 0:
3:       y += x
4:       x -= 1
5:   return y
```

- a) Escribir el control flow graph
b) Escribir los dominadores y postdominadores de cada nodo del control flow graph

Ejercicio 7

Sea el siguiente programa:

```
def factorial(n: int) -> int:
1:     if n == 0:
2:         return 1
3:     else:
4:         result:int = 1
5:         for i in range(1, n+1):
6:             result *= i
7:         return result
```

- Escribir el control flow graph
- Escribir los dominadores y postdominadores de cada nodo del control flow graph
- Escribir el grafo de control dependencia

Ejercicio 8

Sea el siguiente programa:

```
def is_palindrome(s: str) -> bool:
1:     i:int = 0
2:     j:int = len(s) - 1
3:     while i < j:
4:         if s[i] != s[j]:
5:             return False // Target
6:         i += 1
7:         j -= 1
8:     return True
```

- Escribir el control flow graph
- Escribir los dominadores y postdominadores de cada nodo del control flow graph
- Escribir el grafo de control dependencia
- Calcular las distancias en el grafo de control dependencia al target (approach level) de los siguientes inputs

s	Distancia (approach level)
"a"	
"ab"	
"aba"	
"abc"	
"	